Buổi 08 (3 tiết) – Chương 3: Export/Props + Bài tập/Lab (3.7–3.11)

1.1 3.7 Export Component

Để sử dụng một component ở các file khác trong ứng dụng của bạn, bạn cần phải **export** nó. React Native sử dụng hệ thống module của JavaScript (ES Modules) để quản lý việc import và export các thành phần.

Có hai cách chính để export một component:

1. **Named Export (Xuất theo tên):** Bạn export một hoặc nhiều component bằng cách sử dụng từ khóa exporttrước định nghĩa của component. Khi import, bạn cần sử dụng chính xác tên đã export và đặt trong dấu ngoặc nhọn {}.

   Ví dụ:

   ``javascript // Greeting.js import React from 'react'; import { Text } from 'react-native';

   export const Greeting = (props) => { return <Text>Xin chào, {props.name}!</Text>; };

   export const Farewell = (props) => { return <Text>Tạm biệt, {props.name}!</Text>; }; ``

   Cách import:

``javascript import React from 'react'; import { View } from 'react-native'; import { Greeting, Farewell } from './Greeting';

const App = () => { return (<View> <Greeting name="Alice" /> <Farewell name="Bob" /> </View>); };

export default App; ``

2. **Default Export (Xuất mặc định):** Bạn export một component duy nhất làm giá trị mặc định của module. Khi import, bạn có thể đặt bất kỳ tên nào bạn muốn cho component này (không cần dấu ngoặc nhọn). Thông thường, mỗi file component chỉ nên có một default export.

   Ví dụ:

   ``javascript // WelcomeScreen.js import React from 'react'; import { View, Text, StyleSheet } from 'react-native';

   const WelcomeScreen = (props) => { return (<View style={styles.container}> <Text style={styles.title}>Chào mừng đến với ứng dụng!</Text> <Text>{props.message}</Text> </View>); };

   const styles = StyleSheet.create({ container: { flex: 1, justifyContent: 'center', alignItems: 'center', }, title: { fontSize: 24, fontWeight: 'bold', marginBottom: 10, }, });

   export default WelcomeScreen; ``

   Cách import:

   ``javascript import React from 'react'; import { View } from 'react-native'; import Welcome from './WelcomeScreen'; // Có thể đặt tên khác tùy ý

   const App = () => { return (<View> <Welcome message="Đây là nội dung chào mừng." /> </View>); };

   export default App; ``

Trong hầu hết các trường hợp, bạn sẽ sử dụng **default export** cho các file chứa một component chính.

1.2 3.8 Truyền tham số cho Component

Props (viết tắt của properties) là cách để truyền dữ liệu từ một component cha xuống component con. Props là các đối số được truyền vào component (giống như tham số của một hàm) và cho phép component con nhận và sử dụng dữ liệu từ bên ngoài. Props là **read-only** bên trong component con, nghĩa là component con không được phép sửa đổi giá trị của props mà nó nhận được.

Cách truyền props:

Khi sử dụng một component con bên trong JSX của component cha, bạn có thể truyền props bằng cách đặt tên thuộc tính và gán giá trị cho chúng, giống như các thuộc tính HTML.

Ví dụ:

Sử dụng lại component Greeting đã định nghĩa trước đó:

``javascript // App.js import React from 'react'; import { View } from 'react-native'; import Greeting from './Greeting';

const App = () => { return (<View> <Greeting name="Alice" /> {/* Truyền prop 'name' với giá trị "Alice" */} *<Greeting name="Bob" /> {/* Truyền prop 'name' với giá trị "Bob" */} </View>); };

export default App; ``

Cách nhận props trong Functional Component:

Functional component nhận props như là đối số đầu tiên của hàm. Bạn có thể truy cập các prop bằng cách sử dụng cú pháp object destructuring cho tiện lợi.

``javascript // Greeting.js (Functional Component) import React from 'react'; import { Text } from 'react-native';

const Greeting = ({ name }) => { return <Text>Xin chào, {name}!</Text>; };

export default Greeting; ``

Cách nhận props trong Class Component:

Class component nhận props thông qua thuộc tính this.props.

``javascript // Greeting.js (Class Component) import React, { Component } from 'react'; import { Text } from 'react-native';

class Greeting extends Component { render() { return <Text>Xin chào, {this.props.name}!</Text>; } }

export default Greeting; ``

Bạn có thể truyền bất kỳ kiểu dữ liệu JavaScript nào qua props, bao gồm strings, numbers, booleans, objects, arrays, và thậm chí cả functions hoặc các component khác. Việc sử dụng props một cách hiệu quả là rất quan trọng để xây dựng các ứng dụng React Native linh hoạt và có khả năng tái sử dụng cao.

3.9 Câu hỏi ôn tập

8. Export default và named export khác nhau ra sao?

7. Trình tự render lại (re-render) xảy ra khi nào?

6. Tại sao nên tách component nhỏ? Lợi ích về tái sử dụng và bảo trì.

5. Giải thích khái niệm controlled component với TextInput.

4. useEffect chạy khi nào? Nêu 2 trường hợp sử dụng phổ biến.

3. State dùng để làm gì? Khi nào cần state?

2. Props dùng để làm gì? Ví dụ tình huống dùng props.

1. Component là gì? Phân biệt functional component và class component.

3.10 Bài tập thực hành

4. Tách ít nhất 4 component nhỏ cho 1 màn hình (Header, Body, Item, Footer).

3. Tạo form nhập tên + email (TextInput) và validate đơn giản (rỗng/định dạng).

2. Tạo Counter component có state tăng/giảm; hiển thị số lần bấm.

1. Tạo component ProductCard nhận props (name, price) và hiển thị theo layout card.

3.11 Phiếu bài tập thực hành (Mẫu 04)

Sinh viên thực hiện theo yêu cầu, chụp màn hình/ghi video demo và nộp mã nguồn theo hướng dẫn của giảng viên.

Tên bài thực hành	Chương 3 – Lab 03: Xây dựng bộ component tái sử dụng
Mục tiêu	Áp dụng props/state/hooks để xây dựng các component tái sử dụng và form nhập liệu.

Thời lượng gợi ý	120 phút
Chuẩn bị	Dự án RNMultiScreen (hoặc dự án mới). Ôn lại props/state/useEffect.
Nhiệm vụ thực hành	Tạo folder components/ và xây dựng 4 component: AppHeader, PrimaryButton, InputField, Card. Dùng các component đó để xây dựng màn hình Register (tên, email, mật khẩu). Validate: rỗng, email có '@', mật khẩu >= 6 ký tự. Hiển thị lỗi dưới ô nhập (màu đỏ) và disable nút Submit khi chưa hợp lệ.
Sản phẩm nộp	Repo + ảnh màn hình Register hợp lệ/không hợp lệ. Mã nguồn tách component rõ ràng.
Tiêu chí đánh giá	Component tái sử dụng, props hợp lý (30%). Validate hoạt động đúng (30%). UI rõ ràng, code sạch (25%). Cấu trúc thư mục chuẩn (15%).